

Target Detection Algorithm - FCOS

Target Detection Algorithm - FCOS

1. Environment Preparation
2. How to Run
3. Expected Results

1. Environment Preparation

This example is based on the `fcos` model and implements a target detection algorithm function for local video streams. Users can preview the detection results through a monitor.

- Connect the development board and the monitor with an HDMI cable
- Power on the development board and log in through the command line
- Prepare a video file (H264) as input

2. How to Run

To run this case, you need to turn off the screen display first

```
sudo systemctl start lightdm
```

The example code is provided in source form and needs to be compiled with the `make` command before running. The steps are as follows:

```
sunrise@ubuntu:~$ cd /app/cdev_demo/bpu/src
sunrise@ubuntu:/app/cdev_demo/bpu/src$ sudo make
sunrise@ubuntu:/app/cdev_demo/bpu/src$ cd bin
sunrise@ubuntu:/app/cdev_demo/bpu/src/bin$ sudo ./sample -f
/app/model/basic/fcos_512x512_nv12.bin -m 1 -i 1080p_.h264 -w 1920 -h 1080
```

Parameter configuration:

- -f: Model file path
- -h: Height of the input video
- -w: Width of the input video
- -i: Path of the input video
- -m: Model type, default is 1

If you want to turn the screen back on, enter `sudo systemctl start lightdm`

3. Expected Results

After the program runs correctly, it will output the video and the rendered image after algorithm detection through the `HDMI` interface. Users can preview it through the monitor.

The running log is as follows:

```
sunrise@ubuntu:/app/cdev_demo/bpu/src/bin$ sudo ./sample -f
/app/model/basic/fcos_512x512_nv12.bin -m 1 -i 1080p_.h264 -w 1920 -h 1080
```

```

[BPU_PLAT]BPU Platform Version(1.3.1)!
[HBRT] set log level as 0. version = 3.14.5
[DNN] Runtime version = 1.9.7_(3.14.5 HBRT)
Model info:
model_name: fcos_512x512_nv12Input count: 1input[0]: tensorLayout: 2 tensorType:
1 validShape:(1, 3, 512, 512, ), alignedShape:(1, 3, 512, 512, )
Output count: 15Output[0]: tensorLayout: 0 tensorType: 13 validShape:(1, 64, 64,
80, ), alignedShape:(1, 64, 64, 80, )
Output[1]: tensorLayout: 0 tensorType: 13 validShape:(1, 32, 32, 80, ),
alignedShape:(1, 32, 32, 80, )
Output[2]: tensorLayout: 0 tensorType: 13 validShape:(1, 16, 16, 80, ),
alignedShape:(1, 16, 16, 80, )
Output[3]: tensorLayout: 0 tensorType: 13 validShape:(1, 8, 8, 80, ),
alignedShape:(1, 8, 8, 80, )
Output[4]: tensorLayout: 0 tensorType: 13 validShape:(1, 4, 4, 80, ),
alignedShape:(1, 4, 4, 80, )
Output[5]: tensorLayout: 0 tensorType: 13 validShape:(1, 64, 64, 4, ),
alignedShape:(1, 64, 64, 4, )
Output[6]: tensorLayout: 0 tensorType: 13 validShape:(1, 32, 32, 4, ),
alignedShape:(1, 32, 32, 4, )
Output[7]: tensorLayout: 0 tensorType: 13 validShape:(1, 16, 16, 4, ),
alignedShape:(1, 16, 16, 4, )
Output[8]: tensorLayout: 0 tensorType: 13 validShape:(1, 8, 8, 4, ),
alignedShape:(1, 8, 8, 4, )
Output[9]: tensorLayout: 0 tensorType: 13 validShape:(1, 4, 4, 4, ),
alignedShape:(1, 4, 4, 4, )
Output[10]: tensorLayout: 0 tensorType: 13 validShape:(1, 64, 64, 1, ),
alignedShape:(1, 64, 64, 1, )
Output[11]: tensorLayout: 0 tensorType: 13 validShape:(1, 32, 32, 1, ),
alignedShape:(1, 32, 32, 1, )
Output[12]: tensorLayout: 0 tensorType: 13 validShape:(1, 16, 16, 1, ),
alignedShape:(1, 16, 16, 1, )
Output[13]: tensorLayout: 0 tensorType: 13 validShape:(1, 8, 8, 1, ),
alignedShape:(1, 8, 8, 1, )
Output[14]: tensorLayout: 0 tensorType: 13 validShape:(1, 4, 4, 1, ),
alignedShape:(1, 4, 4, 1, )
libiar: hb_disp_set_timing done!
dispaly init ret = 0
vps open ret = 0
module bind vps & display ret = 0
display start ret = 0
[x3_av_open_stream]:[380]:probesize: 5000000
decode start ret = 0
module bind decoder & vps ret = 0
[ERROR] ["vdec"][video/src/vdec_group.c:348]
[8870.450264]vdec_channel_bump_thread[348]: VDEC_MODULE module try again

[draw_rect]:[137]:=====point is 0,return=====
fps:55.555556,processing time:18

```